

UNCERTAINTY-GUIDED CLOSED-LOOP SYNTHETIC GENERATION DOES NOT IMPROVE SYNTHETIC-TO-REAL CAMOUFLAGED OBJECT DETECTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Camouflaged object detection is constrained by the cost of mask annotation, which motivates training on synthetic imagery from camouflage generators. If generation is cheap, a model’s own uncertainty could steer *where* the next images are generated, closing a loop between training and data synthesis. We test that idea. Instantiating the loop in a mean-teacher synthetic-to-real framework with a published camouflage generator, we fix the decision rule before training and run 24 models across two architectures and three seeds. Targeted allocation does not beat uniform allocation: the gap is within noise on both architectures and both endpoints. We dissect the failure into four measured causes and separate those specific to this framework and generator — a per-epoch optimisation budget that cannot grow with the dataset, and an exhausted foreground supply whose objects are copied rather than synthesised — from those concerning the data and the signal, which travel further: the target distribution’s best silhouette coefficient is 0.1600, and removing the uncertainty information while preserving the allocation’s shape reproduces the same effect. A fifth candidate cause, too narrow a conditioning channel, is refuted by source reading and withdrawn. Independently, we find that 41 of 76 images in a standard COD benchmark are re-encoded copies of training data, invisible to exact hashing, and we release a detector and a clean evaluation protocol.

1 INTRODUCTION

Camouflaged object detection asks a model to segment objects whose appearance has evolved to defeat segmentation. Ground-truth masks are correspondingly expensive, and the field is small enough that most results rest on a handful of datasets [CITE: COD10K] [CITE: NC4K]. Generative models offer an appealing way out: a camouflage generator can synthesise a concealing background behind any supplied foreground [CITE: LAKE-RED], and synthetic-to-real training then adapts from generated data to real photographs [CITE: S2R-COD].

If generation is cheap, the natural next question is not *how much* to generate but *where*. A trained model already knows which parts of the target distribution it handles badly; that signal could steer the next batch of generations toward exactly those regions, and the loop could be closed — train, measure deficiency, generate where deficient, retrain. The idea is well motivated: it is the standard active-learning argument [CITE: uncertainty sampling] [CITE: generative active learning] applied to a generator instead of an annotator, in a domain where annotation is the binding constraint. We expected it to work.

It does not. We instantiate the loop in a mean-teacher synthetic-to-real framework with a published camouflage generator, fix a decision rule in advance, and train 24 models across two architectures and three seeds. Targeted allocation does not beat uniform allocation: the gap is within noise on both architectures and both endpoints (Section 5).

A null is only worth reporting if it comes with a mechanism and an honest account of its reach. We therefore separate our claims into three tiers and hold to the distinction throughout. **T1** findings are

properties of this training loop or this generator; **T2** findings concern the target data or the uncertainty signal and travel beyond our instantiation; **T3** findings are independent of the method entirely. The distinction is not decoration: our most striking cause — that the training loop’s optimisation budget cannot grow with the dataset — is T1, and reading it as evidence that synthetic data cannot help would be unsupported by anything we measured.

Contributions.

- **A pre-registered null.** Uncertainty-guided closed-loop generation yields no measurable accuracy gain in this setting. The decision rule was committed before the first training run and is reported unchanged, including the finding that the campaign resolved a coarser effect than it set out to (Section 5).
- **A four-cause dissection, tiered by scope.** We localise the failure to a pinned optimisation budget (T1), an exhausted foreground supply (T1/T2), diffuse target structure (T2), and an uncertainty signal that adds little beyond concentration (T2); a fifth candidate, a conditioning bottleneck, is refuted by source reading and withdrawn (Section 6).
- **A target distribution with little to target.** The target pool’s best silhouette coefficient is 0.1600 and stays below 0.25 in all three embedding spaces we tried — a property of the data, not of our trainer, and a precondition any cluster-allocation method needs and does not get here (Section 6).
- **Severe contamination in a standard COD benchmark, and a clean protocol.** Independently of the above, 41 of 76 CHAMELEON images (53.9%) are re-encoded copies of training-pool images, invisible to exact hashing. We release a detector, the contaminated file list, and an evaluation protocol (Section 7).

2 RELATED WORK

Camouflaged object detection and its benchmarks. COD is evaluated on a small number of datasets — COD10K, NC4K, CAMO and CHAMELEON — with architectures such as SINet, SINet-v2 and SegMaR [CITE: COD10K] [CITE: NC4K] [CITE: CAMO] [CITE: CHAMELEON] [CITE: SINet] [CITE: SINet-v2] [CITE: SegMaR]. Because these sets are reused across nearly all published comparisons, their internal relationships matter for every reported number; Section 7 shows one such relationship has gone unnoticed.

Synthetic data for camouflage. Annotating camouflaged objects is unusually expensive, which motivates generating training data. Camouflage-specific generators paint a concealing background behind a supplied foreground [CITE: LAKE-RED] [CITE: camouflage generation other], typically built on diffusion inpainting [CITE: diffusion inpainting]. Synthetic-to-real pipelines then adapt from generated to real camouflage, of which the framework we test is a recent instance [CITE: S2R-COD]. We test one generator inside one framework; other generators are related work, not evidence about them.

Deciding what data to acquire. Choosing where to spend a labelling or generation budget is the subject of active learning: uncertainty sampling, coverage and core-set criteria, and more recently budget allocation over generative models [CITE: active learning survey] [CITE: uncertainty sampling] [CITE: core-set selection] [CITE: generative active learning]. Our stage (ii) is an uncertainty criterion and stage (iii) an allocation rule, so the approach under test is an active-learning policy over a generator rather than over an annotator. Self-training with a mean teacher supplies the underlying adaptation signal [CITE: mean teacher] [CITE: domain adaptation seg].

Negative results and benchmark integrity. Reporting methods that do not work is undersupplied relative to its value, and pre-registration has been proposed to make such reports credible [CITE: negative results venue] [CITE: pre-registration in ML]. Separately, a line of work documents train–test contamination and near-duplicate leakage in widely used datasets [CITE: train-test contamination] [CITE: dataset duplication]. Our second contribution belongs to that line and, unlike our first, does not depend on the method under test.

3 THE APPROACH UNDER TEST

We specify the approach in full because our contribution is a precise account of why it fails; each component below is later shown to be a distinct point of breakdown. Section 6 maps each component to the measurement that localises its contribution.

3.1 SETTING

The base framework is cross-domain self-training with a mean-teacher objective (CSRDA) for camouflaged object detection [CITE: S2R-COD]. A labelled *source* pool of synthetic camouflage images is paired with an unlabelled *target* pool of real photographs. A student network is trained by gradient descent; a teacher is maintained as an exponential moving average of the student with momentum $\lambda = 0.996$ and is never back-propagated. Each optimisation step consumes one source batch and one target batch. The objective is

$$\mathcal{L} = \mathcal{L}_{\text{sup}} + \mathcal{L}_{\text{ES}}, \quad \mathcal{L}_{\text{ES}} = 0.9 \mathcal{L}_{\text{EA}} + 0.3 \mathcal{L}_{\text{SW}}, \quad (1)$$

where $\mathcal{L}_{\text{sup}}$ is supervised loss on the labelled source batch, $\mathcal{L}_{\text{EA}}$ is a Sobel edge-alignment term between student and teacher predictions on the target batch, and $\mathcal{L}_{\text{SW}}$ is a saliency-weighted cross-entropy against the teacher’s prediction [CITE: S2R-COD].

Training runs for two self-training rounds. After the first round, a confidence-based selection step (CLS) admits target images whose student–teacher edge disagreement falls below a fraction of the running mean,

$$\text{edge_loss}(x) < u \cdot \overline{\text{edge_loss}}, \quad u = 0.8, \quad (2)$$

assigns each admitted image a pseudo-label by thresholding the teacher’s class activation map at $\tau = 0.4$, and appends the resulting pairs to the source pool for the second round. Model selection takes the teacher checkpoint minimising validation MAE on a held-out real set, which is never used as an evaluation endpoint.

3.2 THE IDEA: CLOSING THE LOOP ON THE MODEL’S OWN UNCERTAINTY

The synthetic source pool is produced by a background-generation model that paints a novel camouflaging background behind a supplied foreground and its mask [CITE: LAKE-RED]. Because generation is cheap relative to annotation, it is natural to ask *where* in the target distribution the next batch of generations should be spent. The approach under test answers this with the model’s own error signal, in four stages.

(i) Target clustering. The unlabelled target pool is embedded with a frozen vision backbone and partitioned by k -means into k clusters. We use DINOv2-L at 518px and $k = 75$, selected by maximising the mean silhouette coefficient over $k \in \{5, 10, 15, 20, 30, 50, 75, 100, 150\}$ [CITE: DINOv2] [CITE: silhouette]. Embeddings are L2-normalised once at load time.

(ii) Deficiency scoring. Each cluster receives a scalar deficiency score, its mean per-image edge disagreement es under Eq. 1, computed on the *target* images and requiring no ground truth. Scoring the target side rather than a labelled endpoint is what makes the loop self-contained.

(iii) Budget allocation. A generation budget of B images is distributed over clusters by a softmax of the deficiency scores at temperature $T = \alpha \cdot \text{sd}(es)$,

$$w_j \propto \exp\left(\frac{es_j - \max_i es_i}{T}\right), \quad T = \alpha \cdot \text{sd}(es), \quad (3)$$

converted to integers by the largest-remainder method so the allocation sums to exactly B . We set $\alpha = 1.0$, the scale-free operating point at which the temperature equals the spread of the scores themselves. At $B = 1000$ and $\alpha = 1.0$ the allocation funds all 75 clusters with a maximum share of 0.194.

(iv) **Foreground selection and generation.** Within each funded cluster, foregrounds are served in order of cosine similarity to the cluster centroid, and a greedy pass fills each cluster’s quota, displacing requests that cannot be met from the available foreground supply. The selected foregrounds and their masks are passed to the generator, which returns one new image per foreground. Selection is deterministic: it involves no random draw, so the targeted condition is identical across training seeds.

3.3 CONDITIONS

Isolating the effect of *where* generations are spent requires holding constant everything else that a larger pool changes. We therefore compare four source pools that share the same base pool, verified byte-identical by per-file SHA-256 against a provenance manifest rather than assumed:

- **A0** — base pool only. The condition comparable to the published baseline, and *not* a clean control for the others: at a fixed optimisation budget a smaller pool gives every image more exposure per epoch (Section 6).
- **A2** — base pool plus B duplicates of the same foregrounds selected by condition B, carrying their *original* backgrounds. This matches pool size and foreground identity while removing the generated background, isolating generation itself.
- **B** — base pool plus B generated images for B foregrounds drawn uniformly without replacement.
- **C** — base pool plus B generated images for the B foregrounds chosen by stages (i)–(iv). Identical injection mechanics to B; only the *selection* differs.

$\Delta(C - B)$ is therefore the quantity of interest: targeted against uniform allocation at equal pool size, equal budget, equal foreground supply and equal mask provenance. $\Delta(B - A2)$ separately isolates the generated background.

3.4 WHAT IS HELD IDENTICAL

Beyond the base pool, the campaign fixes the architecture, the method flag, the number of self-training rounds, the epoch and batch schedule, the learning rate and decay, the loss coefficients of Eq. 1, the seed governing initialisation and shuffling, the unlabelled target pool, the model-selection set and rule, the evaluation protocol, the generation budget B , and the backbone initialisation, each asserted rather than assumed.

One quantity is *not* held constant by this design, and it is the reason the null has a mechanism rather than merely an absence: the number of gradient steps per epoch is set by the shorter of the two data loaders and is therefore pinned by the target pool, independent of how large the source pool becomes. We return to this in Section 6.

4 EXPERIMENTAL SETUP AND PRE-REGISTRATION

Endpoints. The primary endpoint is COD10K-test (2026 images), reported as the structure measure S_α [CITE: S-measure] [CITE: COD10K]. NC4K (4121 images) is a secondary endpoint: it is reported but never decides [CITE: NC4K]. CHAMELEON is *withdrawn* as an endpoint, for reasons established independently of this campaign and reported in Section 7. CAMO (250 images) is the model-selection set and is never an endpoint.

Runs. Two architectures (SINet, SINet-v2) $\times$ four conditions $\times$ three seeds $\{42, 43, 45\}$ give 24 training runs, each of two self-training rounds. Because the campaign compares conditions rather than tuning them, the schedule is fixed by the code and cannot drift from the command line; per-architecture values are in Appendix A.

The decision rule, fixed before the first run. The rule below was committed before any training began and never edited afterwards; the governing document states that the campaign is void and must be re-run if any part of it changed once a single S_α had been computed. Let $\hat{\sigma}$ be the pooled

Table 1: **Main result.** S_α per condition, mean $\pm$ standard deviation over seeds {42, 43, 45}; higher is better. $\Delta(C - B)$ is the pre-registered claim and $2\hat{\sigma}$ its decision threshold; “sign” in parentheses counts seeds agreeing in direction. All values at four decimal places. The frozen decision rule, the per-run metrics and the machine-readable verdict are in the supplementary material; Appendix C maps every value to its measurement-log entry.

Arch.	Endpoint	A0	A2	B	C	$\Delta(C - B)$	$2\hat{\sigma}$
SINet	COD10K [†]	0.7010	0.7077	0.7134	0.7185	+0.0050 (3/3)	0.0179
SINet	NC4K	0.7581	0.7632	0.7669	0.7690	+0.0021 (3/3)	0.0083
SINet-v2	COD10K [†]	0.6891	0.6921	0.6951	0.6947	−0.0004 (2/3)	0.0123
SINet-v2	NC4K	0.7444	0.7479	0.7486	0.7496	+0.0009 (2/3)	0.0105
VERDICT: WITHIN NOISE IN ALL FOUR CELLS							
Per-arm s.d., primary cell: A0 0.0173, A2 0.0019, B 0.0018, C 0.0041.							
[†] primary endpoint. $ \Delta \leq 2\hat{\sigma}$ in every row.							

within-arm standard deviation of S_α across all four conditions and three seeds ($df = 8$), estimated separately for each architecture and endpoint. For a gap Δ between two conditions:

REAL EFFECT	$\Delta > 2\hat{\sigma}$ and sign consistent in 3/3 seeds
WITHIN NOISE	$ \Delta \leq 2\hat{\sigma}$
REAL REGRESSION	$\Delta < -2\hat{\sigma}$ and sign consistent in 3/3 seeds
INCONCLUSIVE	$ \Delta > 2\hat{\sigma}$ but sign not 3/3 — reported as-is; seeds are <i>not</i> added

No p -value is computed at $n = 3$. The claim under test is $\Delta(C - B)$; $\Delta(B - A2)$ is reported alongside it, and gaps against A0 are reported for comparability with the published baseline only.

Gates before any comparison. The pre-flight stage asserted 16 conditions and all passed, including that every condition’s base pool matches the provenance manifest byte for byte, that the targeted condition reproduces the allocation of the earlier selection experiment exactly, that the two generated conditions overlap only at the chance rate, and that the endpoint-measured signal appears nowhere in the pipeline. The scoring code was itself validated against a previously committed measurement before being used to decide anything.

5 RESULT: NO MEASURABLE GAIN

Table 1 reports the campaign. **The claim under test, $\Delta(C - B)$, is WITHIN NOISE in all four architecture $\times$ endpoint cells.** Targeting the generation budget at the model’s least-well-served clusters does not beat spending it uniformly, by the rule fixed before the first run.

Two features of the table deserve comment before any interpretation. First, the arm means *do* increase monotonically from A0 through A2 and B to C in three of the four cells, and the ordering is the one the approach predicts — it is simply smaller than the noise the design was built to see through. Second, the largest single gap anywhere in the campaign is not the one under test.

The one significant gap is inadmissible, and we say so rather than report it. Across all 20 gap tests the campaign returns WITHIN NOISE eighteen times, INCONCLUSIVE once, and REAL EFFECT once. The single REAL EFFECT is A0 $\rightarrow$ C on SINet/NC4K, $\Delta = +0.0109$, consistent in 3/3 seeds. It is disqualified twice over: it sits on the secondary endpoint, which the pre-registered rule states never decides, and it is measured against A0, which is not a clean control because at a pinned optimisation budget its smaller pool gives every image more exposure per epoch (Section 6). We report it for completeness and draw nothing from it.

Power: the design did not reach the resolution it declared. The pre-registration stated that the design “resolves an effect $\sim$ half the paper’s MT $\rightarrow$ Ours gap (0.0142) and no smaller”. The realised noise estimate on the primary endpoint is $2\hat{\sigma} = 0.0179$, which *exceeds* that reference gap:

Table 2: **Four measured causes, with scope, and one refuted.** T1 is specific to this framework or generator; T2 concerns the data or signal and generalises further. A T1 cause is *not* evidence about generators-in-the-loop in general. Row (ii), refuted by source reading, is withdrawn and carries no tier. Roman numerals index the paragraphs of Section 6; Appendix C gives the measurement-log entry for each row.

Cause	Measured quantity	Tier
(i) Optimisation budget is pinned	Steps per epoch fixed at 253 / 127 in both rounds of all 24 runs, independent of pool size	T1
(ii) Generator conditioning width — <i>refuted, withdrawn</i>	Not a bottleneck: the foreground latent also reaches the denoiser at full resolution, where global self-attention can read it from any background position	—
(iii) Foreground pool is exhausted	Generated set is a bijection onto the 4447 existing foregrounds; object pixels are copied, not synthesised	T1/T2
(iv) Target clusters are diffuse	Best silhouette 0.1600; below 0.25 in all three embedding spaces	T2
(v) Signal adds little over shape	Removing the targeting information but keeping the allocation shape reproduces the same separation	T2

the campaign resolves roughly $1.3\times$ it, not half of it. **The null is therefore a null at a coarser resolution than intended, and both halves of that sentence belong in any report of this result.** The rule anticipated this: it fixes the bar at $2\hat{\sigma}$ as measured, not at an assumed noise level, so a larger $\hat{\sigma}$ raises the bar rather than invalidating the test. Two quantities that would sharpen the statement are unavailable and are not estimated here: run-to-run seed variance, which would require retraining that was deferred, and any rate translating a pool shift into an S_α change, which we found could not be re-derived from the available runs and therefore do not assume.

6 WHY IT FAILS

A null is only useful if it comes with a mechanism. We measured four, and they do not have the same reach. A fifth, a conditioning bottleneck in the generator, we advanced and withdraw in paragraph (ii). Recalling the scope tiers of Section 1: **T1** findings are properties of this training loop or this generator, **T2** findings concern the data or the signal and travel beyond our instantiation, and **T3** is independent of the method entirely. Table 2 summarises; the tier column is the load-bearing one.

(i) Added data cannot buy optimisation. [T1] Each step consumes one source and one target batch, and the loop stops when the shorter loader is exhausted. Because the target pool is fixed, the number of gradient steps per epoch is pinned — measured at 253 (SINet) and 127 (SINet-v2) in *both* rounds of all 24 runs, regardless of how much synthetic data was added. The consequence is *dilution, not exclusion*: both loaders shuffle, so the added images are sampled — they occupy about 18.4% of the first round’s gradient content — but every image’s expected exposure per epoch falls correspondingly. A condition with the base pool alone gives each image 0.9103 exposures per epoch against 0.7432 for the enlarged pools, some 22% more. This is the cause most easily over-read. It says that *this* loop converts extra data into a rearranged mixture rather than more learning; it says nothing about whether generated data helps a training regime that scales its optimisation with dataset size. It also explains why A0 is not a clean control, and hence why A2 exists.

(ii) The conditioning channel — a cause we withdraw. [T1, refuted by source reading] A selection method can only matter if the difference it selects for survives into the render, and we advanced a narrow conditioning channel as the reason it might not: before the foreground conditions background synthesis it is reduced to 16 superpixel means of a three-channel latent — 48 scalars — which would bound what any selection policy could express. Reading the generator’s source refutes that account. The summary is real but is one route among several. The conditioning tensor handed to the diffusion denoiser retains the foreground latent at full resolution over the object region and enters as input channels of a U-Net whose self-attention is unmasked and global, so any background

position can read the foreground directly; a second route admits the same full-resolution latent into every regenerated position through a 1×1 convolution inside the summarising module. Foreground information is therefore not confined to the summary, and we withdraw the cause. It does not follow that the generator makes *semantic* use of what it receives — that needs a foreground perturbation with the summary held fixed, which we did not run — nor that it is steerable (Section 8). The withdrawal removes a T1 explanation and leaves more of the account to (iv) and (v).

(iii) There are no new foregrounds to find. [T1 mechanism, T2 consequence] Every image the loop can add is a re-rendering of a foreground already in the pool. The generated set is a bijection onto the 4447 source foregrounds: 4447 of 4447 trace back, none originates outside, and none is left unrendered. Moreover the generator composites the original object pixels back into its output, so the object is reproduced rather than re-imagined — mean absolute error inside the object is 5.603 against 70.802 in the background (a ratio of 12.64), and not one of the 4447 images has an object-interior error above 40. Exhaustion here is therefore an identity statement about pixels, not a statistical claim about distributional overlap that could be argued about. The mechanism is specific to a generator that composites the object back; the consequence — that re-rendering an exhausted foreground supply adds no new object information — applies to any pipeline whose novelty is confined to backgrounds.

(iv) The target distribution has little cluster structure to target. [T2] The method assumes the target pool decomposes into clusters that can be individually under-served. It barely does. The best mean silhouette coefficient over $k \in \{5, \dots, 150\}$ is 0.1600, at $k = 75$ — far below the 0.25 that would indicate usable structure — and the finding is robust across embedding spaces: 0.1465 for DINOv2-L at 224px and 0.0568 for CLIP-L, with the peak below 0.25 in every space. We report a companion failure honestly: in the CLIP space the silhouette does not attain an interior maximum at all but falls monotonically to 0.0357 at $k = 150$, so “choose k by silhouette” is not even well posed there. This is a property of the target data, not of our trainer: an allocation policy over clusters is only as meaningful as the clustering it allocates over.

(v) The uncertainty signal contributes little beyond concentration. [T2] The targeted and uniform conditions do produce measurably different pools. But an audit of *why* they differ found that the difference is not attributable to the uncertainty signal: destroying the targeting information while preserving the *shape* of the allocation — how concentrated it is, not where it points — reproduces the same separation. Of seven declared attribution thresholds, two passed and five failed, and the honest verdict is that the separation is real but is produced by concentration rather than by targeting. This matters more than the null itself: it means the negative result is not merely about a weak generator, but about a signal that, in this setting, carries little information a concentration prior does not already carry.

Where the synthetic images sit, and a metric that cannot tell us. [T2] It is tempting to explain the null by saying the synthetic pool is simply far from the real target distribution. Stated absolutely, that explanation does not survive its own controls. A linear probe separates real from generated images at an area-under-curve of 0.9993 — but re-encoding the *identical* real images as JPEG at quality 30 separates them from themselves at 0.9928, higher than either genuine real-versus-real control (0.9381 and 0.9225). The declared vacuity flag fired: at that operating point the metric is not measuring distributional distance. What does survive is comparative and stated as coverage: a genuinely different real camouflage dataset covers the target manifold at 0.79–0.90, real photographs of an unrelated genre at 0.71–0.75, and the synthetic images generated *from those very photographs* at 0.13–0.54. We state the limit of this explicitly: it is a characterisation of the distribution, not a bound on training utility. A distant or low-coverage synthetic set can still improve a model, and in our own campaign the condition means rose monotonically. A3 supplies a mechanism consistent with the absence of a resolvable gain; it does not show that no gain exists.

7 BENCHMARK CONTAMINATION AND A CLEAN EVALUATION PROTOCOL

This section is independent of everything above. It would stand if the method under test had worked, and it is why CHAMELEON is absent from Table 1. **Of the 76 images in CHAMELEON, 41 (53.9%) are re-encoded copies of images in the training pool a COD10K-trained model has already seen** — 40 in COD10K-train and one in CAMO.

Table 3: **Contamination against the COD10K training pool.** “Exact” counts byte- or decoded-pixel-identical files, “re-enc.” counts confirmed re-encodings at tolerance 6/255; the two are disjoint by construction. The nearest-neighbour gap is tolerance-independent. Lower panel: the CHAMELEON count as the confirmation tolerance varies. Appendix C gives the measurement-log entry for each row.

Benchmark	n	Exact	Re-enc.	Total (%)	NN gap
CHAMELEON	76	0	41	41 (53.9)	41 below 5.51, next 40.58 ($7.36\times$)
COD10K-test	2026	7	2	9 (0.44)	none — continuous
NC4K	4121	0	1	1 (0.02)	none — continuous
CAMO [‡]	250	0	4	4 (1.6)	none — continuous
NC4K vs COD10K- <i>test</i>	4121	0	0	0 (0.0)	none; closest 20.61, $3.44\times$ tol.
CHAMELEON count at tolerance 1/2/3/5/6: 11 / 26 / 37 / 40 / 41 (saturates)					
[‡] model-selection set, never an endpoint. Rows other than CHAMELEON are measured against on-disk copies, not author-sourced ones; treat them as lower bounds.					

Exact hashing declares the set clean. Hashing CHAMELEON against the training pool returns *zero* collisions at the file-byte level and zero at the decoded-pixel level. The duplicates are re-encodings: same photograph, different compression. Any audit that stops at hashing — as audits usually do — will certify a 53.9%-contaminated benchmark as uncontaminated. That is the trap this section exists to close.

Detection. Images are grouped by exact pixel dimensions, since a re-encode preserves them. Within a group, each image is reduced to a 32×32 greyscale descriptor and pairs are shortlisted by descriptor RMS distance; survivors are decoded at full resolution and confirmed when the mean absolute difference over RGB falls below a tolerance of 6/255. We deliberately do *not* contrast-normalise the descriptor: normalising destroys the scale that discriminates a re-encode from a merely similar photograph, and it is precisely what caused our own first pass to report 10 of 76 rather than 41. The criterion is a *mean*, not an exact match: the confirmed pairs reach maximum per-pixel differences as high as 70, so any test demanding pixel equality rejects all of them.

The count does not depend on the tolerance. Sweeping the confirmation tolerance over $\{1, 2, 3, 5, 6\}$ gives 11, 26, 37, 40, 41 contaminated images: the count saturates rather than growing with permissiveness. Independently of any tolerance, the sorted nearest-neighbour distances show a gap — 41 images lie below 5.51 and the next is at 40.58, a factor of 7.36 — so the contaminated set is separated from the rest of the benchmark by an order of magnitude, not by a threshold we chose. Varying the shortlist depth changes nothing.

The finding survives being re-tested on data we did not control. Our first audit used the CHAMELEON copy in our own working tree. Because that is exactly the weakness such a claim invites, we repeated it against the release distributed by the dataset’s authors. The two copies are byte-identical in all 76 files, and the re-audit returns the same 41 of 76. It also corrected three of our own earlier values — we report those corrections rather than quietly adopting them.

One declared check failed, and we leave it failing. We had declared that every confirmed pair should show differing JPEG quantisation tables as independent evidence of re-encoding. It does not hold: two files in the CHAMELEON release are PNGs carrying a `.jpg` extension and so have no quantisation table at all, and our original comparison had read a *missing* table as a differing one — absence of evidence counted as evidence. The corrected tally is 40 of 41 pairs evidenced by quantisation table and one by container format (413 KB PNG against a 132 KB JPEG), so every pair retains independent evidence, but the check as worded fails and we did not reword it after the fact.

What we do *not* claim: inflated scores. The obvious next step is to argue that reported CHAMELEON numbers are inflated. We measured this and cannot support it. Removing the seven exact duplicates from COD10K-test moves MAE by 1.2×10^{-5} , and in the wrong direction; the

duplicated images sit at the 48th percentile of the clean set’s error distribution. On CHAMELEON, the contaminated and clean subsets are indistinguishable on every proxy we tried, and the direction of the difference *flips* depending on which of the two circulating mask releases is used to score. **The conclusion is about independence, not inflation: a contaminated column cannot be interpreted as an independent measurement, regardless of whether it happens to be higher.**

A second, larger source of incomparability. While reconciling the two CHAMELEON mask releases we found that the choice of release alone changes MAE by a factor of 2.7 on *identical* predictions (0.2196 against 0.0816), with only 27 of 76 masks identical between them. Papers reporting CHAMELEON without naming the mask release are not comparable with each other even setting contamination aside.

A clean protocol. For a COD10K-trained camouflaged-object model we recommend: report COD10K-test and NC4K, disclosing that both were audited against on-disk copies and that a fraction of each is unchecked; do not report CHAMELEON as an independent endpoint, and if it is reported at all, report it on the named 35-image uncontaminated subset together with the mask release used; never report the set used for model selection as an endpoint; and run the detector on your own directories before reporting anything. NC4K in particular survives the additional check its use invites — it does not overlap COD10K-test at any level, with the closest approach $3.44\times$ the confirmation tolerance. The detector, the list of contaminated filenames, and the protocol are included in the supplementary material. Two caveats are ours to carry: unchecked is not clean, so every rate in Table 3 is a lower bound; and only the CHAMELEON row has been verified against author-sourced data, so the others remain provisional.

8 CONCLUSION

Uncertainty-guided closed-loop generation does not improve synthetic-to-real camouflaged object detection in the setting we tested. The gap between targeted and uniform allocation is within noise on two architectures and two endpoints under a rule fixed before training, and the campaign resolved a coarser effect than it was designed to. We hold the scope of that result where the measurements put it: the two causes with the largest apparent force — a per-epoch optimisation budget that cannot grow with the dataset, and a foreground supply whose objects are copied rather than synthesised — are properties of this loop and this generator. What generalises further is less flattering to the idea itself: the target distribution has little cluster structure to allocate over, and the uncertainty signal contributes little beyond the concentration of the allocation it induces.

What a working system would require. The causes imply concrete requirements rather than general prohibitions. *First*, a training regime whose optimisation scales with dataset size, so that added data buys gradient steps instead of diluting exposure; under a fixed step budget no acquisition policy can express itself. *Second*, a source of genuinely new foregrounds. Our null is correctly read as “targeting does not help once the foreground pool is exhausted”, and the experiment that would separate exhaustion from targeting — a condition supplied with new foregrounds from a salient-object dataset, holding the generator fixed — is designed and deferred rather than run, because admitting a pool that has not been swept for leakage to the standard of Section 7 into a load-bearing comparison would repeat the error that section documents. *Third*, a deficiency signal aligned with what the task actually rewards. Ours is an edge-disagreement measure, and its own attribution audit found the targeting information largely redundant given the allocation shape. *Fourth*, a generator that can be steered toward a named deficiency rather than only supplied with a foreground — a requirement about the *shape* of the interface, not its capacity. Ours carries foreground information at full resolution (Section 6(ii)) but accepts only that foreground and its mask, with no cross-attention port through which a deficiency could be named. Whether a generator offering one would help is untested.

What survives regardless. The contamination result does not depend on any of the above. A benchmark in routine use is 53.9% re-encoded training data, and exact hashing certifies it as clean. The detector, the contaminated file list, and the evaluation protocol are released, and we expect them to outlast the negative result they were found alongside.

AI USE STATEMENT

(This section is **required** and does not count toward the page limit.)

We used a large language model in three roles, and disclose each. *Analysis.* An LLM was used to query, cross-check and summarise the measurement artifacts described in Appendix C — for example, to recompute per-condition means from the committed per-run metrics and to check reported figures against the measurement log. It did not perform any of the measurements themselves: every number in this paper was produced by a committed script and written to the log before being quoted. *Drafting and editing.* An LLM assisted in drafting and revising the manuscript text, including the structure of Sections 3–8 and the tables, working from the committed artifacts and a written plan. *Not used.* We did not use generative AI to produce research ideas, to generate or synthesise any data or experimental result, or to write the experiment code whose outputs this paper reports; the training, audit and detection code predates and is independent of the drafting process. We reviewed all AI-assisted work: every quantitative claim was checked against its source artifact, and claims we could not source are marked as unverified in Appendix D rather than asserted. We take responsibility for the final content of this work, including all text, claims and artifacts.

ETHICS STATEMENT

This work reports that a widely used evaluation benchmark substantially overlaps a standard training split, and releases the means to detect it. We note three considerations. *First*, the finding concerns dataset construction, not conduct: the overlap is invisible to exact hashing, so a benchmark can be assembled and used in good faith without anyone noticing, and we attribute no fault to the dataset’s authors or to any group that has reported results on it. *Second*, we deliberately do not convert the finding into a claim about any published number. We measured whether contamination inflates reported scores and found no detectable effect (Section 7); asserting that specific results are inflated would go beyond our evidence and could unfairly impugn other work. *Third*, the released material — a detector and a list of filenames already present in a public dataset — introduces no new personal data and involves no human subjects. We release it because the alternative, reporting a contamination rate without the means to reproduce or extend the check, would ask the community to take the claim on trust. The method we test is a third party’s; we test one instantiation of it, scope our conclusions accordingly, and report the negative result without implying error on the part of its authors.

REPRODUCIBILITY STATEMENT

The experimental setup, the pre-registered decision rule and the gates asserted before any comparison are given in Section 4; per-architecture schedules, backbone and embedder identifiers, and the exposure calculation are in Appendix A. Appendix C maps every quantity reported in the paper to the entry in the measurement log that produced it, and Appendix D states the threshold accounting and lists every quantity we could not verify. The detection procedure of Section 7 is described in enough detail to reimplement, and the implementation, its self-test and the affected filenames are provided in the supplementary material together with the measurement log, the per-run metrics and the frozen decision rule.

A IMPLEMENTATION DETAILS

Schedules. Hyperparameters are set in code after argument parsing and cannot be overridden from the command line, so they are identical across all conditions. SINet: 40 epochs declared and 39 executed, batch 16, learning rate 10^{-4} decayed by 0.1 at epoch 30, no gradient clipping. SINet-v2: 100 declared and 99 executed, batch 32, learning rate 10^{-4} decayed by 0.1 at epoch 50, gradient clipping enabled.

Steps and exposure. Steps per epoch are 253 (SINet) and 127 (SINet-v2), giving 4048 and 4064 source images seen per epoch. Per-image expected exposure per epoch is 0.9103 for the base-only condition against 0.7432 for the enlarged pools on SINet, and 0.9139 against 0.7461 on SINet-v2.

Backbones and embedders. Backbone initialisations are pinned by file size and SHA-256 (resnet50-11ad3fa6.pth, 102,540,417 bytes; res2net50_v1b_26w_4s-3cf99910.pth, 103,197,949 bytes). Embeddings use vit_large_patch14_dinov2.lvd142m at 224 and 518px and vit_large_patch14_clip_224.openai, all frozen, with features L2-normalised at use.

Object-compositing measurement, both pools. Mean absolute error inside the object against the background was 5.603 vs 70.802 (ratio 12.64) for the locally regenerated pool and 9.633 vs 70.555 (ratio 7.32) for the pool the trainer actually reads, over all 4447 images of each, with zero objects exceeding an interior error of 40.

Deferred by design. Resize interpolation, the numerical stabiliser in the allocation softmax, tie-breaking in the greedy selection, and the Sobel kernels of Eq. 1 are unchanged from the released implementations of the framework and generator and are not restated here.

B CLUSTER QUALITY ACROSS k AND EMBEDDING SPACE

Table 4: **Mean silhouette coefficient** over k -means partitions of the 4040-image target pool, by embedding space and k . The peak of each row is in bold. No space exceeds 0.25; in CLIP the curve has no interior maximum, falling across the grid, so selecting k by silhouette is not well posed there. Full sweep artifacts are in the supplementary material; see Appendix C.

Embedding space	$k=5$	10	15	20	30	50	75	100	150
DINOv2-L / 518px	0.0566	0.0923	0.1152	0.1326	0.1527	0.1556	0.1600	0.1557	0.1507
DINOv2-L / 224px	0.0538	0.0868	0.1053	0.1180	0.1378	0.1465	0.1457	0.1419	0.1367
CLIP-L / 224px	0.0568	0.0550	0.0486	0.0486	0.0488	0.0448	0.0396	0.0382	0.0357

C TRACEABILITY

Every quantity in this paper originates in an append-only measurement log supplied with the supplementary material. One entry is written per accepted run and records the command, the environment, the declared thresholds with their outcomes, and the artifacts produced; entries are never edited or deleted, so superseded runs remain visible alongside the ones reported. Table 5 maps each reported group of quantities to its entry.

D MEASUREMENT DISCIPLINE AND WHAT REMAINS UNVERIFIED

Across the log blocks cited in this paper, 143 thresholds were declared in advance, 127 passed and 16 failed. Failures are reported rather than repaired: several are the findings themselves (the contamination rate, the instability of the binary cluster-structure label), two are corrections to our own earlier work, and one — the quantisation-table check of Section 7 — was left failing rather than reworded after the fact.

The following are *not* established by any committed measurement. Except where a reading of source is named as the basis, they are asserted nowhere in this paper:

- Whether the generator makes semantic use of the foreground information it receives. That the architecture does *not* confine that information to the 48-scalar summary we establish by reading the generator’s source rather than by measurement (Section 6(ii)); the experiment that would show whether the available capacity is *used* — perturbing the foreground while holding the summary values fixed and measuring the generated background — was not run. We assert no bound in either direction.
- Any inflation of reported scores attributable to contamination. We measured this and found no detectable effect; the conclusion is about independence, not inflation.

Table 5: **Where each reported quantity comes from.** Entry labels are those used in the supplied measurement log. Where several entries share a label, the one cited is named; earlier entries are retained but superseded. One row reports a reading of the generator’s source rather than a measurement and so has no log entry; the files read are hashed in the supplementary material.

Reported in	Quantity	Log entry
Section 4, gates	16 pre-flight assertions	ABC #1
Table 1; Section 5	condition means, $\hat{\sigma}$, gaps, verdicts	ABC #3
Section 6(i); App. A	steps per epoch, exposure	ABC #2
Section 6(ii)	conditioning-path architecture	source reading; no entry
Section 6(iii); App. A	foreground bijection, compositing error	D1
Section 6(iv); Table 4	silhouette across k and space	B1 #3
Section 6(v)	attribution audit	C1 #4
Section 6, last paragraph	coverage, probe controls, vacuity flag	A3
Table 3, CHAMELEON row	contamination rate, tolerance sweep	D2 #5; re-audit D2R #4
Table 3, NC4K vs COD10K-test	independence of the two endpoints	D2_NC4K #2
Section 7, mask releases	MAE under each mask release	D2R #4
App. D	threshold accounting	all entries cited above

- Run-to-run seed variance, and any rate translating a pool shift into a change in S_{α} . Both would require training that was deferred; the latter could not be re-derived from the available runs.
- Author-sourced verification of the contamination rates for COD10K-test, NC4K and CAMO. Only the CHAMELEON row has been re-audited against data we did not control. [TODO: verify — re-audit owed.]
- Whether any published table reports the contaminated benchmark, which we cannot check from the artifacts available to us.
- An overlap we measured but did not log to the standard the rest of this work requires: a further 32 of the 76 CHAMELEON images appear to coincide with COD10K-*test*. [TODO: verify — present in the artifact, absent from the measurement log; used in no claim above.]